

# Diploma Thesis

## Investigation of Machine Vibration Analysis System Design

Vasileios Dimitriou

Department of Electrical and Software Engineering  
University of Thessaly

September, 2025



## ① Introduction

## ② Background

## ③ Engineering and Design

## ④ Implementation

## ⑤ Results

## ⑥ Conclusions

# 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

## 5 Results

## 6 Conclusions

# Predictive Maintenance and Condition Monitoring

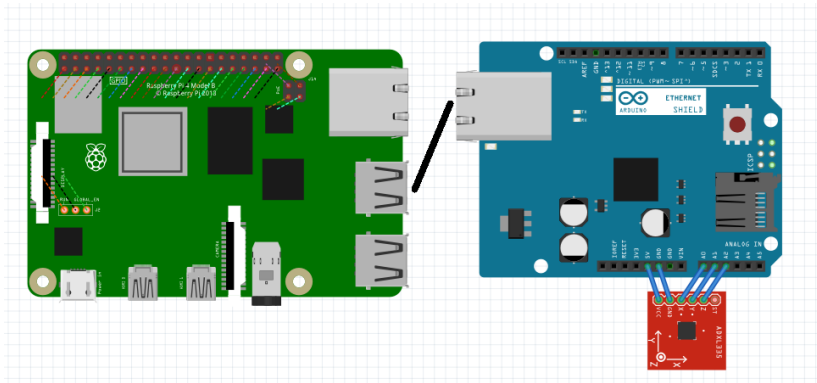


Figure 1: Hardware architecture with signal flow.

# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].

# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].
- Development of software for data acquisition and transmission from Arduino UNO R3 to Raspberry Pi 4.

# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].
- Development of software for data acquisition and transmission from Arduino UNO R3 to Raspberry Pi 4.
- Development of software for FFT-based spectral analysis software on Raspberry Pi 4 for frequency-domain feature extraction.

# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].
- Development of software for data acquisition and transmission from Arduino UNO R3 to Raspberry Pi 4.
- Development of software for FFT-based spectral analysis software on Raspberry Pi 4 for frequency-domain feature extraction.
- Development of a secure, authenticated API client for structured data transmission from Raspberry Pi 4 to Supabase.



# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].
- Development of software for data acquisition and transmission from Arduino UNO R3 to Raspberry Pi 4.
- Development of software for FFT-based spectral analysis software on Raspberry Pi 4 for frequency-domain feature extraction.
- Development of a secure, authenticated API client for structured data transmission from Raspberry Pi 4 to Supabase.
- Implementation of a Vercel API endpoint that fetches measurement data from Supabase database.

# Predictive Maintenance and Condition Monitoring

- Development of a low-cost embedded system for predictive maintenance [see Figure 1].
- Development of software for data acquisition and transmission from Arduino UNO R3 to Raspberry Pi 4.
- Development of software for FFT-based spectral analysis software on Raspberry Pi 4 for frequency-domain feature extraction.
- Development of a secure, authenticated API client for structured data transmission from Raspberry Pi 4 to Supabase.
- Implementation of a Vercel API endpoint that fetches measurement data from Supabase database.
- Development of a React frontend that requests and visualizes data from the Vercel API.

## 1 Introduction

## 2 Background

Fast-Fourier Transform  
Fault Detection

## 3 Engineering and Design

## 4 Implementation

## 5 Results

## 6 Conclusions

## 1 Introduction

## 2 Background

### Fast-Fourier Transform

### Fault Detection

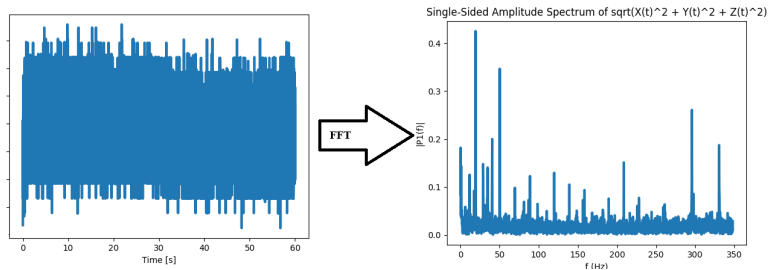
## 3 Engineering and Design

## 4 Implementation

## 5 Results

## 6 Conclusions

# FFT



**Figure 2:** Conversion of the time-domain signal into its frequency-domain power representation using the FFT.

## 1 Introduction

## 2 Background

Fast-Fourier Transform  
Fault Detection

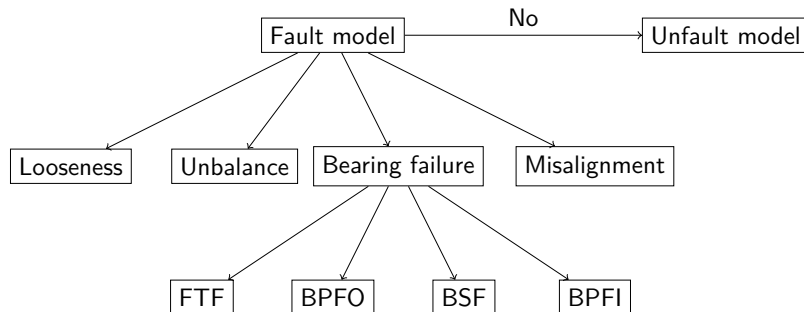
## 3 Engineering and Design

## 4 Implementation

## 5 Results

## 6 Conclusions

# Machinery Fault Tree Analysis



**Figure 3:** Fault tree analysis diagram showing the relationship between machine conditions and specific fault frequencies.

1 Introduction

2 Background

3 Engineering and Design

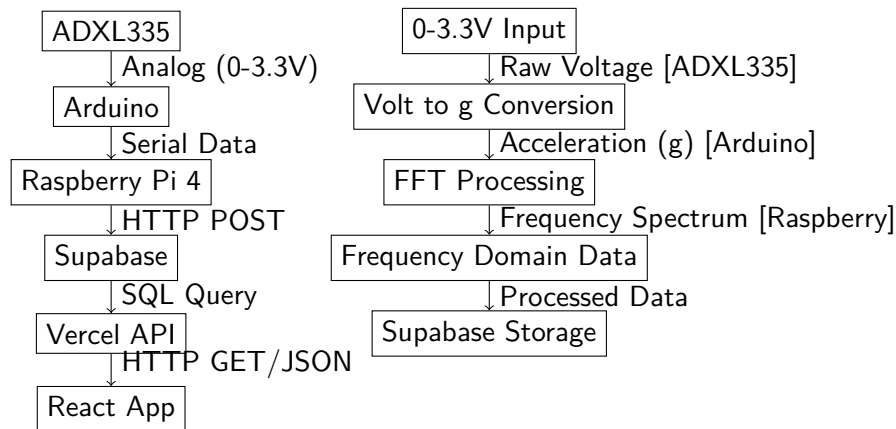
4 Implementation

5 Results

6 Conclusions



# System Architecture and Dataflow



**Figure 4:** Complete system architecture and data flow and transformation pipeline from physical sensor to web visualization dashboard.

# System boundaries

- The ADXL335 measurement range is  $\pm 3g$ .

# System boundaries

- The ADXL335 measurement range is  $\pm 3g$ .
- The Arduino UNO R3 sampling rate is 700 [Hz].

# System boundaries

- The ADXL335 measurement range is  $\pm 3g$ .
- The Arduino UNO R3 sampling rate is 700 [Hz].
- Flash memory and SRAM limitations require data to be transmitted in batches.

# System boundaries

- The ADXL335 measurement range is  $\pm 3 g$ .
- The Arduino UNO R3 sampling rate is 700 [Hz].
- Flash memory and SRAM limitations require data to be transmitted in batches.
- The frequency-domain analysis is limited to 350 Hz (Nyquist limit).

# System boundaries

- The ADXL335 measurement range is  $\pm 3 g$ .
- The Arduino UNO R3 sampling rate is 700 [Hz].
- Flash memory and SRAM limitations require data to be transmitted in batches.
- The frequency-domain analysis is limited to 350 Hz (Nyquist limit).
- The measurement time window determines the total number of samples collected.

## 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

Arduino UNO Algorithm

Raspberry Pi Algorithm

Vercel Implementation and Supabase Integration

## 5 Results

## 6 Conclusions

## 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

Arduino UNO Algorithm

Raspberry Pi Algorithm

Vercel Implementation and Supabase Integration

## 5 Results

## 6 Conclusions



# Arduino UNO Algorithm

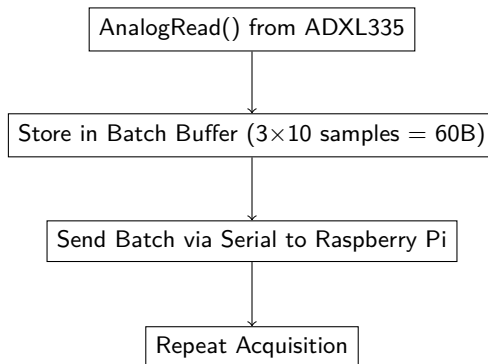


Figure 5: The Arduino UNO Algorithm.

## 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

Arduino UNO Algorithm

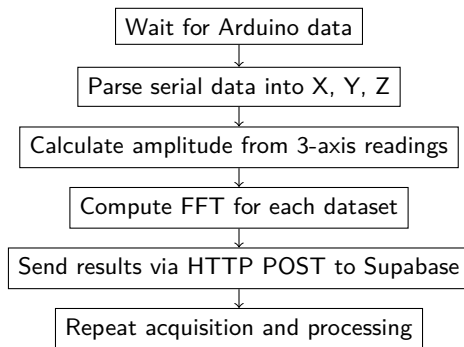
Raspberry Pi Algorithm

Vercel Implementation and Supabase Integration

## 5 Results

## 6 Conclusions

# Raspberry Pi Algorithm



**Figure 6:** The Raspberry Pi 4 Algorithm: serial data reception, FFT processing, and cloud transmission.

## 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

Arduino UNO Algorithm

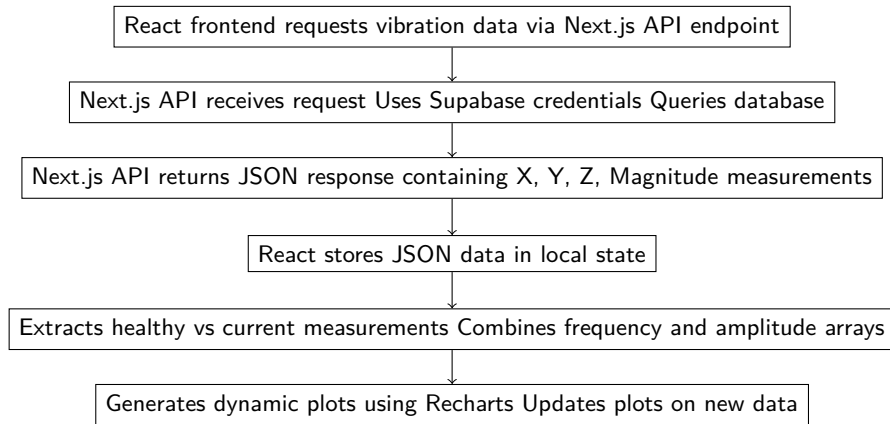
Raspberry Pi Algorithm

Vercel Implementation and Supabase Integration

## 5 Results

## 6 Conclusions

# Vercel and Supabase Algorithmic Flow



**Figure 7:** Flow diagram of the Vercel and Supabase algorithm: from frontend request to dynamic visualization.

1 Introduction

2 Background

3 Engineering and Design

4 Implementation

5 Results

6 Conclusions

# System Verification and Validation

- End-to-end checks were performed across the entire system: data acquisition from ADXL335 sensors via Arduino UNO, serial transmission to Raspberry Pi 4, processing and posting to Supabase, and frontend visualization in Vercel/React.

# System Verification and Validation

- End-to-end checks were performed across the entire system: data acquisition from ADXL335 sensors via Arduino UNO, serial transmission to Raspberry Pi 4, processing and posting to Supabase, and frontend visualization in Vercel/React.
- Stress tests confirmed reliable operation under expected data loads.



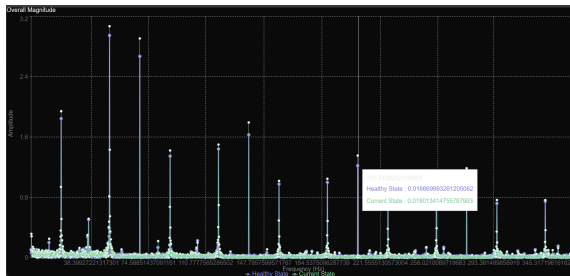
# System Verification and Validation

- End-to-end checks were performed across the entire system: data acquisition from ADXL335 sensors via Arduino UNO, serial transmission to Raspberry Pi 4, processing and posting to Supabase, and frontend visualization in Vercel/React.
- Stress tests confirmed reliable operation under expected data loads.
- Verified no data loss, correct timing, and accurate real-time plotting.

# System Verification and Validation

- End-to-end checks were performed across the entire system: data acquisition from ADXL335 sensors via Arduino UNO, serial transmission to Raspberry Pi 4, processing and posting to Supabase, and frontend visualization in Vercel/React.
- Stress tests confirmed reliable operation under expected data loads.
- Verified no data loss, correct timing, and accurate real-time plotting.
- System performance validated for FFT processing, transmission rates, and visualization updates.

# System Verification and Validation



**Figure 8:** Visualization of the overall vibration magnitude. This chart combines all three coordinate axes into a single representation, showing the total vibration energy with the healthy baseline (blue) compared against the current state (green).

## 1 Introduction

## 2 Background

## 3 Engineering and Design

## 4 Implementation

## 5 Results

## 6 Conclusions

# System Overview and Achievements

- Developed and validated an end-to-end condition monitoring system for bearing fault detection using vibration analysis.

# System Overview and Achievements

- Developed and validated an end-to-end condition monitoring system for bearing fault detection using vibration analysis.
- Integrated hardware (ADXL335, Arduino UNO, Raspberry Pi 4) with cloud services (Supabase, Vercel) for data acquisition, processing, and visualization.

# System Overview and Achievements

- Developed and validated an end-to-end condition monitoring system for bearing fault detection using vibration analysis.
- Integrated hardware (ADXL335, Arduino UNO, Raspberry Pi 4) with cloud services (Supabase, Vercel) for data acquisition, processing, and visualization.
- FFT-based comparative analysis of X, Y, Z axes and magnitude against healthy baselines for fault identification.

# System Overview and Achievements

- Developed and validated an end-to-end condition monitoring system for bearing fault detection using vibration analysis.
- Integrated hardware (ADXL335, Arduino UNO, Raspberry Pi 4) with cloud services (Supabase, Vercel) for data acquisition, processing, and visualization.
- FFT-based comparative analysis of X, Y, Z axes and magnitude against healthy baselines for fault identification.
- **Arduino collects and transmits triaxial acceleration in batches at 700 [Hz] sampling rate without loss; Raspberry Pi converts to frequency-domain and posts to Supabase.**



# System Overview and Achievements

- Developed and validated an end-to-end condition monitoring system for bearing fault detection using vibration analysis.
- Integrated hardware (ADXL335, Arduino UNO, Raspberry Pi 4) with cloud services (Supabase, Vercel) for data acquisition, processing, and visualization.
- FFT-based comparative analysis of X, Y, Z axes and magnitude against healthy baselines for fault identification.
- Arduino collects and transmits triaxial acceleration in batches at 700 [Hz] sampling rate without loss; Raspberry Pi converts to frequency-domain and posts to Supabase.
- Vercel dashboard visualizes frequency spectra interactively for real-time monitoring.